



YZM2031 - Data Structures and Algorithms

Fall 2026

Fettah KIRAN

Department of AI and Data Engineering

fkiran@yildiz.edu.tr

[f-kiran.github.io](https://github.com/f-kiran)

- **Class Hours and Location:** Friday 09:00 - 11:50 @ B020
- **Lab Hours and Location:** Friday 14:00 - 15:50 @ D-B08
- **Fall 2026:** Weeks 1-7 + 9-13 teaching, **Week 8 Midterm**
- **Attendance:** Required by university regulations; attendance is not separately tracked by the instructor
- **Main Textbook:** Weiss, Mark Allen. "Data structures and algorithm analysis in C++." (2022).

Suggested IDEs

Use any of the following environments for compiling, running, and debugging C++:

- **Visual Studio Code (VS Code)** — recommended for the course; available on Windows, macOS, and Linux
- **Code::Blocks** — lightweight, beginner-friendly C++ IDE
- **Xcode** — recommended for students using macOS

Choose one environment and make sure you can compile a small C++ program before the first lab.

Course Plan

Week	Subject	Preparation
1	C++ Programming Basics and Classes	Weiss Ch. 1
2	Lists and Linked List	Weiss Ch. 3.1-3.5
3	Stack and Queue	Weiss Ch. 3.6-3.7
4	Queue and Chars/Strings	Weiss Ch. 3.7
5	Algorithm Analysis	Weiss Ch. 2
6	Trees	Weiss Ch. 4.1-4.2
7	Binary Search Tree	Weiss Ch. 4.3
8	Midterm	
9	Balanced Tree (AVL)	Weiss Ch. 4.4
10	Heap Data Structure	Weiss Ch. 6
11	Sorting Algorithms	Weiss Ch. 7
12	Disjoint Sets	Weiss Ch. 8
13	Hash Tables	Weiss Ch. 5
14	Final	

Why This Course?

Algorithms + Data Structures = Programs

— Niklaus Wirth

- Learn how to organize data for efficient access and modification
- Analyze how an algorithm scales as the input grows
- Implement and compare data structures in C++
- Practice explaining design choices, not only writing code

Learning Outcomes

By the end of the course, you should be able to:

- **Trace** a data structure operation and explain its state changes
- **Implement** core structures such as lists, stacks, queues, trees, and graphs
- **Analyze** time and space complexity using appropriate notation
- **Compare** alternative implementations and choose one for a problem
- **Test and debug** pointer-based C++ code, including boundary cases

Midterm 30% + Final 40% + Coursework 30% = 100%

- Midterm and final exam: face to face, room to be announced
- Coursework: labs and project requirements are defined in the current syllabus
- Any recap bonus follows the current syllabus and cannot exceed the stated course limit

There are 12 labs. Check the current syllabus for submission rules and deadlines.

How We Will Work

- Lectures introduce an idea, then make it concrete with traces and code
- Labs turn the idea into a tested implementation
- Questions may ask you to **predict**, **trace**, **implement**, or **compare**
- Bring questions about code that compiles but produces the wrong result

We assume you already know C programming:

- variables, conditions, loops, and functions;
- arrays, structs, pointers, and pointer arithmetic;
- dynamic allocation and deallocation with `malloc` and `free`;
- compiling, running, and debugging a small program.

In Week 1, we focus on the C++ features used to implement data structures:

- references, `const`, and function overloading;
- `string`, `vector`, and stream-based input/output;
- classes, constructors, methods, access control, and destructors;
- templates and the Standard Template Library;
- C++ ownership: `new` / `delete`, RAI, and smart pointers.

Quick check: reference versus pointer

```
int value = 10;  
int& reference = value;  
reference = 25;
```

What is the value of `value` after the last line? How would the same update look through a C pointer?

The next technical lecture connects these foundations to implementation:

1. A **node** stores data and a link to another node.
2. A **list** organizes nodes into a sequence.
3. Operations such as insert, remove, and search change that structure.
4. We compare implementations by correctness, memory use, and running time.

Next: Lists and Linked Lists

Review C pointers and dynamic memory, then focus on C++ references, classes, and ownership before Week 2.